

```
import pandas as pd
```

✓ Uploading a Dataset

YouTo upload files from your local system to Colab, you can use `files.upload()` from `google.colab`. This will prompt you to select a file from your computer.

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) creditcard_fraud.csv

creditcard_fraud.csv(text/csv) - 915700 bytes, last modified: 6/15/2026 - 100% done
Saving creditcard_fraud.csv to creditcard_fraud.csv

Once the file is uploaded, you can load it into a pandas DataFrame. Assuming you uploaded a CSV file, you can do so like this:

```
df=pd.read_csv("creditcard_fraud.csv")  
df.head()
```

	cardholder_name	card_number	merchant_name	merchant_category	amount
0	Allison Hill	4961023726432058	Merchant: Calhoun, Davis and Richmond	grocery	189.9
1	Noah Rhodes	4997245971557937	Merchant: Powell and Sons	online_shopping	14.5
2	Angie Henderson	4770115491773269	Merchant: Cox- Mejia	dining	23.4
3	Daniel Wagner	4968568888610852	Merchant: Ortiz, Snow and Garner	grocery	30.0
4	Cristian Santos	4358679433644706	Merchant: Barker PLC	grocery	63.5

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 10000 entries, 0 to 9999  
Data columns (total 11 columns):
```

```

#   Column      Non-Null Count  Dtype
---  -
0   cardholder_name    10000 non-null   object
1   card_number         10000 non-null   int64
2   merchant_name       10000 non-null   object
3   merchant_category   10000 non-null   object
4   amount              10000 non-null   float64
5   hour                10000 non-null   int64
6   distance_from_home  10000 non-null   float64
7   is_online           10000 non-null   int64
8   is_foreign          10000 non-null   int64
9   velocity_last_hour  10000 non-null   int64
10  is_fraud             10000 non-null   int64
dtypes: float64(2), int64(6), object(3)
memory usage: 859.5+ KB

```

```
df.drop(columns=['cardholder_name','card_number','merchant_name'],inplace=True)
```

```
tracs=df["is_fraud"].value_counts()
```

Google Gemini

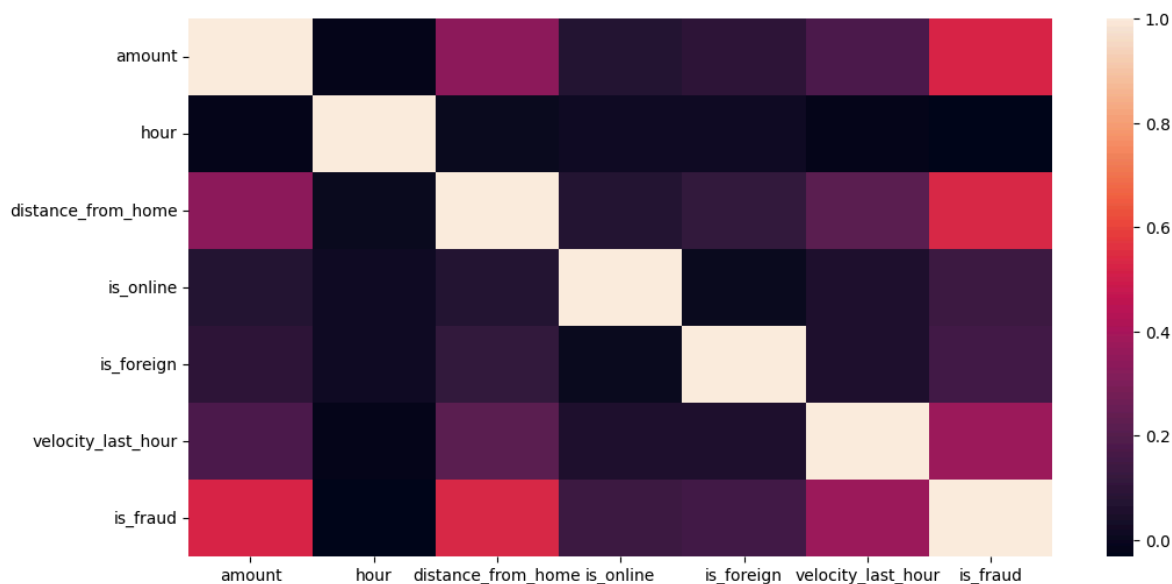
```

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
sns.heatmap(df.corr(numeric_only=True))

```

<Axes: >



```
df=pd.get_dummies(df,columns=['merchant_category'],drop_first=True)
```

```
df.head()
```

	amount	hour	distance_from_home	is_online	is_foreign	velocity_last_hou
0	189.90	6	0.63	1	1	
1	14.50	15	1.12	0	0	
2	23.46	16	27.38	1	0	
3	30.00	6	5.75	1	0	
4	63.56	17	0.42	1	0	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
X=df.drop(columns=['is_fraud'])
y=df['is_fraud']
```

```
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_stat
```

X_train

	amount	hour	distance_from_home	is_online	is_foreign	velocity_last_
3209	18.26	7	6.62	1	0	
6276	41.02	18	18.01	1	0	
8953	23.08	8	2.82	0	0	
4780	8.32	15	19.05	0	0	
2470	105.54	10	4.55	0	0	
...	...	...	...	...	...	...
6652	20.12	11	3.76	1	0	
301	23.91	16	56.63	1	0	
6437	1129.51	2	8.78	1	0	
6212	30.15	13	24.25	0	0	
7439	59.73	17	4.15	1	0	

8000 rows × 11 columns

Next steps:

[Generate code with X_train](#)[New interactive sheet](#)

y_train

	is_fraud
3209	0
6276	0
8953	0
4780	0
2470	0
...	...
6652	0
301	0
6437	1
6212	0
7439	0

8000 rows × 1 columns

dtype: int64

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf=RandomForestClassifier(n_estimators=100,random_state=42,class_weight='bal
```

```
rf.fit(X_train,y_train)
```

▼

RandomForestClassifier

i ?

```
RandomForestClassifier(class_weight='balanced', random_state=42)
```

```
rf_pred=rf.predict(X_test)
```

```
array([0, 0, 0, ..., 0, 0, 0])
```

```
from sklearn.metrics import accuracy_score,confusion_matrix,classification_r
```

```
rf_accuracy=accuracy_score(y_test,rf_pred)
print(rf_accuracy)
```

```
0.9955
```

```
print(classification_report(y_test,rf_pred))
```

```
precision    recall  f1-score   support
```

0	1.00	1.00	1.00	1966
1	1.00	0.74	0.85	34
accuracy			1.00	2000
macro avg	1.00	0.87	0.92	2000
weighted avg	1.00	1.00	1.00	2000

Start coding or [generate](#) with AI.

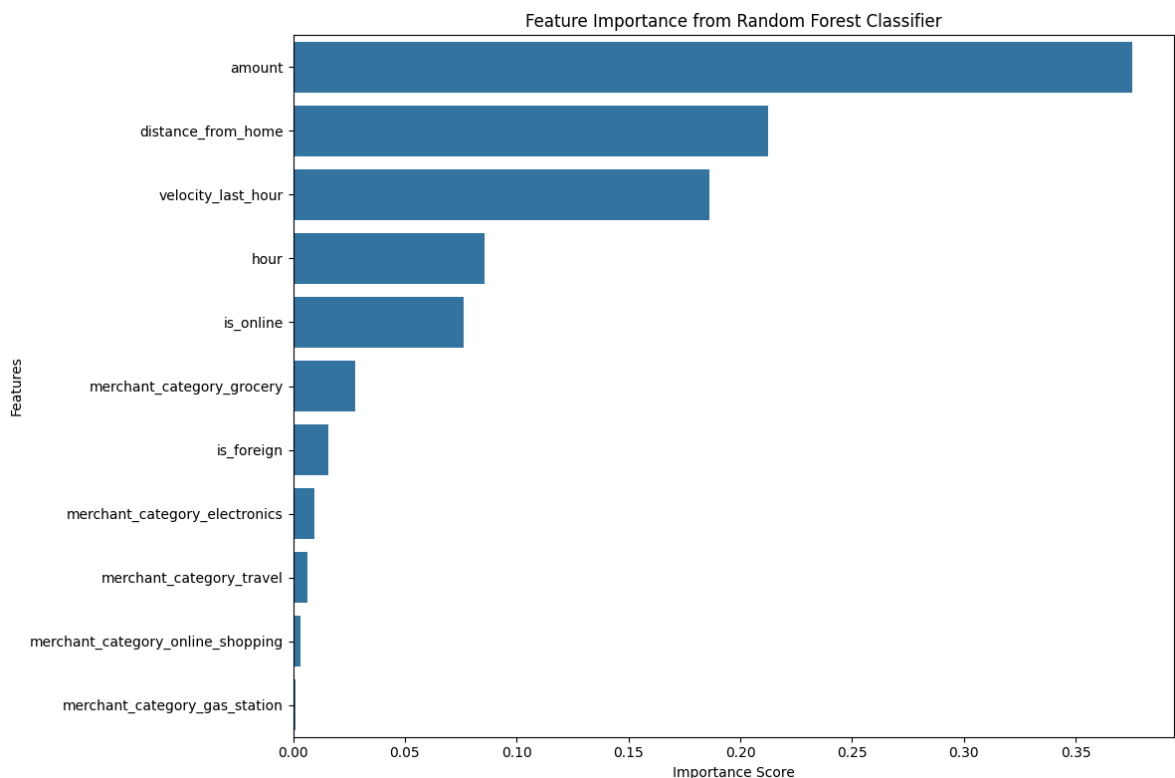
```
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature importances from the trained model
feature_importances = rf.feature_importances_

# Create a pandas Series for better visualization
features = X.columns
importance_df = pd.DataFrame({'Feature': features, 'Importance': feature_imp

# Sort the features by importance
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Plot the feature importances
plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title('Feature Importance from Random Forest Classifier')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.tight_layout()
plt.show()
```



```
import joblib
```

```
joblib.dump(rf, 'ccfd.pkl')
```

```
['ccfd.pkl']
```

```
model1=joblib.load('/content/ccfd.pkl')
```